

1. Pseudocode

PROSEDUR RB-Insert(T, z)

Lakukan standar BST Insert dan beri warna z = MERAH

SELAMA z != T.root DAN z.parent.warna == MERAH

JIKA z.parent == z.parent.parent.left

y = z.parent.parent.right (Paman)

JIKA y.warna == MERAH

// Kasus 1: Paman merah

z.parent.warna = HITAM

y.warna = HITAM

z.parent.parent.warna = MERAH

z = z.parent.parent

LAINNYA

JIKA z == z.parent.right

// Kasus 2: z adalah anak kanan

z = z.parent

LEFT-ROTATE(T, z)

// Kasus 3: z adalah anak kiri

z.parent.warna = HITAM

z.parent.parent.warna = MERAH

RIGHT-ROTATE(T, z.parent.parent)

LAINNYA (Simetris dengan kondisi di atas)

T.root.warna = HITAM

2. Implementasi Kode

class Node:

def __init__(self, data, color="RED"):

self.data = data

self.color = color

self.left = None

self.right = None

self.parent = None

class RedBlackTree:

def __init__(self):

self.NIL = Node(0, color="BLACK")

self.root = self.NIL

def left_rotate(self, x):

y = x.right

x.right = y.left

if y.left != self.NIL:

y.left.parent = x

y.parent = x.parent

if x.parent == None:

self.root = y

elif x == x.parent.left:

x.parent.left = y

else:

x.parent.right = y

y.left = x

x.parent = y

Fungsi insert_fixup dan fungsi lainnya akan mengikuti logika pseudocode

Worst Case = $O(\log n)$

$O(\log n)$ Karena aturan pewarnaan dan rotasi,

tinggi maksimal pohon tidak akan pernah melebihi $2\log(n+1)$.

Operasi pencarian, penyisipan,

dan penghapusan tetap efisien
meskipun data masuk secara urut.

Average Case = $O(\log n)$

Pada distribusi data acak, pohon
akan tetap seimbang secara alami,
mirip dengan BST standar
namun dengan jaminan stabilitas tinggi.